What Target-wise ABA Learning Does on Deterministic Causal Fixtures

Evidence from controlled probes H0–H7b

Working synthesis for discussion with Fabrizio

4 August 2026

Status and purpose. The parts to read are the six findings, the four implications, and the conclusion. Everything before them is reference material supporting those findings, and it reads as a chronological account of what I tested and why: the causal fixtures, the learner input and encoding, the terminology, the configurations, and then the controlled probes. Those probes are labelled H0–H7b as internal shorthand, each is described in the text by its question and design, and all of them are complete and analysed. They state no cross-cutting conclusions, which are collected in the findings instead. I will look at the Causal ABA code and paper closely this weekend.

Abstract

The investigation asks what target-wise ABA Learning does when given only a finite binary table sampled from a known causal data-generating process. The principal process is a binary collider $A \to C \leftarrow B$ with independent stochastic roots and a deterministic AND child $C = A \wedge B$. The graph, mechanisms, population distribution, finite sample, learner-visible encoding, learned ABA framework, and evaluator-only reference are kept separate throughout.

The supporting evidence is a sequence of controlled probes recorded under the labels H0–H7b. It begins with a baseline comparison of the published Greedy and non-deterministic configurations across all targets on one frozen table (H0). Later probes each introduce one controlled question: finite-sample support (H1), an irrelevant trailing variable (H2), background-knowledge order with a leading distractor (H3), brave versus cautious learning (H4–H5), folding-token budget and grounded sample size (H6), and the `relto` versus `sechk` assumption-introduction options (H7). For each probe the document records what it was intended to reveal, its principal signal, and the procedural explanation supported by the generated deltas and Prolog traces.

Six findings are drawn from that evidence. A completed Greedy solution enumerates the distinct positive value patterns, so every measured predictor present on a positive row is retained. Nd folding latches onto the first background-knowledge predictor. Brave nd accepts theories for targets that have no mechanism, and cautious nd refuses them. The folding-token ceiling repeats one search attempt under the exact-value encoding. The search is fixed by the sample's distinct value patterns, and multiplicity affects only runtime cost. The `sechk` option confines a brave solution to one variable and makes a cautious search longer. Four design implications follow, covering the folding strategy, the entailment mode, the table representation, and the folding-token ceiling. The conclusion records cautious nd folding with `relto` assumption introduction as the working default configuration, and identifies inspection of the existing Causal ABA implementation as the next step.

Contents

1 Purpose, scope, and organising question

1.1 Research question

> For a finite binary table generated by independent stochastic roots and deterministic non-root mechanisms, and for each observed variable taken in turn as the learning target: what ABA framework does ABALearn emit under the configurations studied here, and how does that framework relate to the evaluator-only mechanism reference and to the information present in the population and in the sample?

The purpose is to describe how the learner transforms the supplied table into rules, assumptions, and contraries, and to classify each emitted theory against a target-dependent evaluator reference. On the principal AND collider $A \to C \leftarrow B$, only the deterministic child C has a desired learned rule over its observed parents:

```
c(A) :- a_val_1(A), b_val_1(A).
```

The roots A and B are generated by independent stochastic mechanisms. There is no underlying deterministic rule relating either root to the other observed variables, and none should be learned.

Relative to that target-dependent reference, the outcomes distinguished in this document are:

- for a deterministic child: a rule that coincides with the mechanism;
- for a deterministic child: a sample-adequate but non-minimal rule;
- for a deterministic child: a defeasible representation that differs from the mechanism string;
- for a root or child: a rule that holds on the finite sample because a population counterexample is missing from that sample;
- for a root: an assumption construction that satisfies the selected learning semantics while leaving the target underdetermined by its predictors;
- search termination under the configured procedure, including timeout or `completed_no_solution`, which for a stochastic root target may be the intended outcome under the reference above.

These classes separate procedural exit status, finite-sample adequacy, and mechanism alignment under the target-dependent reference.

1.2 Scope

The evidence comes from three small binary fixtures, fixed seed-42 samples, and the learner configurations described below. Learning is target-wise: one frozen table is held fixed, and each observed variable is selected in turn as the learning target. Changing the target changes the positive and negative examples and the background-knowledge encoding of the remaining columns; the underlying rows remain the same sample.

Only the deterministic child has an evaluator-desired deterministic rule over its observed parents. Root-target runs are diagnostics of the same pipeline when the target itself is generated stochastically: they show how finite support, learning semantics, and search budget shape the emitted theory or search outcome when no underlying deterministic rule from the other variables should be learned.

1.3 Object of study

The pipeline under study is target-wise ABA Learning over encoded tabular data. For each target it receives background knowledge and labelled examples only, and it emits an ABA framework of rules, assumptions, and contraries under a selected learning semantics and search configuration.

The evidence recorded here therefore concerns learned rule structure, search behaviour, and the relation of those outputs to the evaluator-only mechanism reference for the fixture under study.

2 Conceptual framework and notation

This section fixes the objects, ABA notions, target-wise encoding, and learner controls used throughout the experimental sequence.

2.1 Seven objects kept separate

The experimental reasoning distinguishes the following objects:

1. a generating DAG G;
2. structural mechanisms for the variables in G;
3. the exact observational population distribution P^*;
4. a finite IID sample $\mathcal{D}_n \sim P^*$;
5. target-specific learner input derived from $\mathcal{D}_n$;
6. the learned ABA framework and its emitted delta (the learned additions relative to the input background knowledge); and
7. evaluator-side judgements about sample adequacy, mechanism alignment, and minimality relative to the target-dependent reference.

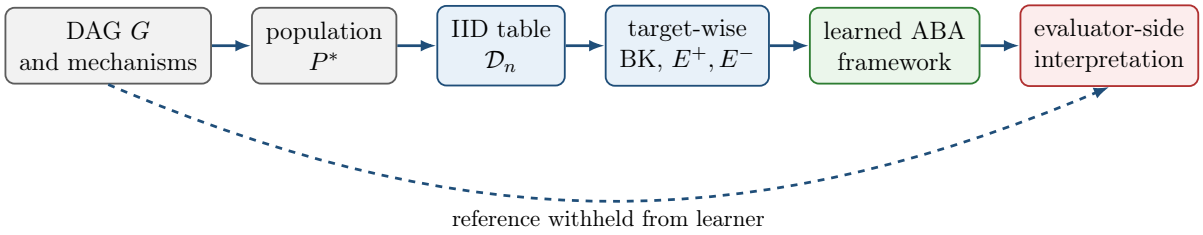


Figure 1: Experimental separation. Only the finite table and its target-wise encoding enter the learner. The generating graph and mechanism reference are reserved for evaluation.

Ordinary faithfulness is a property of the pair (P^*, G). Finite-sample adequacy is a property of a learned framework relative to $\mathcal{D}_n$ under the selected learning semantics. A parent literal appearing in a learned body is a syntactic fact about the emitted delta; any judgement that the delta aligns with the mechanism is an evaluator-side reading.

2.2 ABA framework

An ABA framework is a tuple

$$\mathcal{F} = \langle \mathcal{L}, \mathcal{R}, \mathcal{A}, \bar{\cdot} \rangle,$$

where $\mathcal{L}$ is a language, $\mathcal{R}$ is a set of rules, $\mathcal{A} \subseteq \mathcal{L}$ is a set of assumptions, and $\bar{\cdot}$ maps each assumption $\alpha \in \mathcal{A}$ to its contrary $\overline{\alpha} \in \mathcal{L}$.

A rule has the form

$$h \leftarrow b_1, \ldots, b_m.$$

If $m = 0$, it is a fact. Write $\Delta \vdash_{\mathcal{R}} s$ when a claim $s \in \mathcal{L}$ is derivable from a set of assumptions $\Delta \subseteq \mathcal{A}$ using the rules in $\mathcal{R}$.

Attacks are generated by contraries: Δ attacks an assumption $\alpha \in \mathcal{A}$ when $\Delta \vdash_{\mathcal{R}} \overline{\alpha}$. A set $\Delta \subseteq \mathcal{A}$ is *conflict-free* if it attacks none of its own members. It is *closed* if whenever $\Delta \vdash_{\mathcal{R}} \alpha$ for some $\alpha \in \mathcal{A}$, then $\alpha \in \Delta$.

A set $\Delta \subseteq \mathcal{A}$ is a *stable extension* of $\mathcal{F}$ if it is closed, conflict-free, and attacks every assumption in $\mathcal{A} \setminus \Delta$. Write $\mathrm{Stab}(\mathcal{F})$ for the collection of all stable extensions of $\mathcal{F}$.

A claim $s \in \mathcal{L}$ is accepted with respect to a stable extension Δ if there exists $\Delta' \subseteq \Delta$ such that $\Delta' \vdash_{\mathcal{R}} s$; write $\mathcal{F} \models_{\Delta} s$ when this holds. Write $\mathcal{F} \models_{\mathrm{caut}} s$ when s is accepted with respect to every stable extension of $\mathcal{F}$. A learned framework may therefore combine ordinary rules with explicit exceptional conditions expressed by assumptions and contraries.

2.3 ABA Learning task

An ABA Learning task begins with background knowledge and two finite sets of examples:

$$E^+ = \{e_1^+, \ldots, e_p^+\}, \qquad E^- = \{e_1^-, \ldots, e_q^-\}.$$

The learner applies symbolic transformations to construct a framework $\mathcal{F}'$ under which the examples satisfy the selected consequence condition. The learned object is itself an ABA framework.

The transformations relevant here are:

- **Rote learning.** Add example-specific rules from the positive examples.
- **Folding.** Replace example-specific conditions with reusable BK predicates, producing intensional rules.
- **Subsumption.** Remove a specialised rule when an appropriate more general rule already exists.
- **Assumption introduction.** Make an over-general rule defeasible by adding an assumption, then learn rules for its contrary to represent exceptions.

2.4 Exact-value target-wise encoding

In this investigation the learning task is instantiated from a tabular sample. For each row identifier i, every non-target variable is represented by exactly one value predicate. When c is the target, for example, the BK contains facts such as

```
a_val_1(9).
b_val_0(9).
```

and row 9 contributes either $c(9) \in E^+$ or $c(9) \in E^-$, according to its target value. For a binary target Y,

$$E_Y^+ = \{Y(i) : Y_i = 1\}, \qquad E_Y^- = \{Y(i) : Y_i = 0\}.$$

All non-target columns are available as BK predictors in their serialised order. The encoding interface is the same for every target. The evaluator-side desired object still depends on the target's generating role: only a deterministic child has a desired parent mechanism rule.

2.5 Brave and cautious learning

Brave and cautious learning are the two consequence conditions under which a learned framework $\mathcal{F}'$ may be required to cover E^+ and exclude E^-.

Brave learning requires an existential witness: there must exist some $\Delta \in \mathrm{Stab}(\mathcal{F}')$ that simultaneously accepts every positive example and rejects every negative example,

$$\exists \Delta \in \mathrm{Stab}(\mathcal{F}') \quad \big[\forall e \in E^+,\ \mathcal{F}' \models_{\Delta} e \ \wedge\ \forall e \in E^-,\ \mathcal{F}' \not\models_{\Delta} e\big].$$

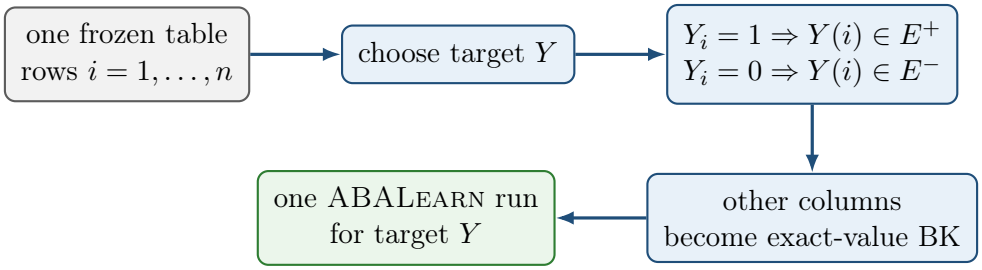


Figure 2: Target-wise encoding. All targets use the same rows; the selected target determines the positive and negative examples and the remaining columns determine the BK.

Cautious learning requires every positive example to be a cautious consequence and every negative example to be excluded as a cautious consequence,

$$\forall e \in E^+,\ \mathcal{F}' \models_{\text{caut}} e, \qquad \forall e \in E^-,\ \mathcal{F}' \not\models_{\text{caut}} e.$$

Thus the two modes quantify differently over $\text{Stab}(\mathcal{F}')$: brave learning uses one witnessing extension, while cautious learning uses acceptance in every stable extension.

2.6 Greedy and non-deterministic folding

Folding is controlled by `folding_mode`, together with companion options for selection and folding space. The two modes used here are as follows.

`folding_mode(greedy).` A rote rule is folded in one deterministic pass against matching background predicates, under `folding_selection(mgr)` and `folding_space(bk)`. This is the folding regime of the AAMAS configuration.

`folding_mode(nd).` Folding proceeds incrementally under a folding-token budget (`folding_steps`), with `folding_selection(any)` and `folding_space(all)`. This is the folding regime of the ECAI configuration and of the cautious nd configurations used later.

Each published configuration also sets learning semantics and `asm_intro`. When two named configurations are compared as wholes, several options change together.

2.7 Assumption introduction: `relto` and `sechk`

Assumption introduction is the transformation that makes an over-general folded rule defeasible by placing an assumption in its body and learning rules for the contrary of that assumption. In ABALEARN it is controlled by the option `asm_intro`, which takes one of two values: `relto` or `sechk`. Both are used after a folded candidate fails the configured post-folding entailment check. They differ in how the first assumption-repair step is chosen.

`asm_intro(relto).` The learner first searches the current framework for an existing assumption that is relative to the body of the folded rule under consideration. If such an assumption is found, it is reused in the repaired rule. If none is found, the learner may generate a new assumption and continue along the subsequent generalisation path.

`asm_intro(sechk).` The learner first generates a new provisional assumption for the folded rule and then subjects the resulting framework to the semantic-checking branch of generalisation (`gen5`/`gen6`), which inspects stable-extension consequences before deciding how the assumption is retained or revised.

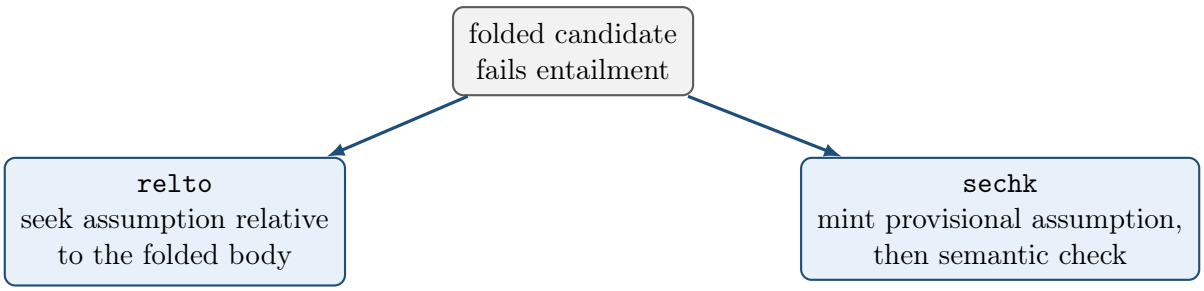


Figure 3: The two `asm_intro` options as alternative first steps of assumption introduction.

3 Causal fixtures and evaluator ground truth

This section records the generating processes, population certificates, evaluator-only mechanism references, and frozen samples used by the probes below. Mathematical labels are A, B, C, D; learner-safe identifiers are `a`,`b`,`c`,`d`. In emitted Prolog rules the capital argument (for example the `A` in `c(A)`) is the row identifier, not the causal variable A.

3.1 Principal fixture: binary AND collider (H0)

The principal fixture is the unshielded collider

$$G_0 : \qquad A \longrightarrow C \longleftarrow B$$

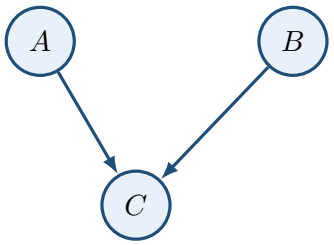
with no A–B edge. The roots are mutually independent,

$$A \sim \text{Bernoulli}(4/5), \qquad B \sim \text{Bernoulli}(7/10), \qquad A \perp\!\!\!\perp B,$$

and the child is the deterministic AND mechanism

$$C = A \wedge B.$$

$$\Pr(A = 1) = 0.8 \qquad \Pr(B = 1) = 0.7$$



$$C = A \wedge B$$

Figure 4: Principal generating DAG (H0). Randomness enters through the independent roots; the non-root mechanism is deterministic.

The induced observational population has four positive-probability atoms:

A	B	C	$P^*(A, B, C)$
0	0	0	$3/50 = 0.06$
0	1	0	$7/50 = 0.14$
1	0	0	$6/25 = 0.24$
1	1	1	$14/25 = 0.56$

The remaining four binary assignments $(0, 0, 1)$, $(0, 1, 1)$, $(1, 0, 1)$, and $(1, 1, 0)$ are structural zeros.

Causal sufficiency is declared by fixture design: the root exogenous noise terms are mutually independent and there is no omitted common cause. The causal Markov condition holds by construction of the joint,

$$P^*(a, b, c) = P^*(a)P^*(b)P^*(c \mid a, b).$$

Ordinary faithfulness was verified exactly by auditing every singleton-pair conditional-independence query against d-separation. The sole population independence is $A \perp\!\!\!\perp B$; conditioning on the collider yields $A \not\perp\!\!\!\perp B \mid C$, and each parent is dependent on C.

Under ordinary conditional-independence semantics, the skeleton is $A - C - B$ and the unshielded collider $A \to C \leftarrow B$ is oriented in the CPDAG. The corresponding MEC contains only G_0. This is the ordinary CI-based reading. Deterministic mechanisms can impose additional functional constraints beyond ordinary CI. The present document stays with the ordinary CI reading. The graph, certificates, and mechanism reference are evaluator-only and are withheld from ABALearn.

3.2 Evaluator-only mechanism reference

For the deterministic child C, the canonical positive-state exact-value rule is

```
c(A) :- a_val_1(A), b_val_1(A).
```

On this fixture the formal truth-table rule, the population-supported rule, and the rule supported by the frozen sample coincide on that string. That string is the evaluator reference against which learned theories for target `c` are compared. Comparison uses rule bodies, assumptions, contraries, and the target's generating role together: a mechanism-aligned string is one form of evidence; an assumption-rich but sample-adequate theory is another object under the same reference.

The roots A and B are generated by independent stochastic mechanisms $A \sim \text{Bernoulli}(4/5)$ and $B \sim \text{Bernoulli}(7/10)$. Neither root is a deterministic function of the other observed variables, so when `a` or `b` is the learning target no underlying deterministic rule from those predictors should be learned.

3.3 Frozen sample for the principal fixture

The principal table is the target-free IID sample with $n = 30$ and seed 42:

Assignment	Count
$(1, 1, 1)$	17
$(1, 0, 0)$	7
$(0, 1, 0)$	4
$(0, 0, 0)$	2

All four population atoms appear. Under the binary exact-value encoding the implied example counts are

$$a : 24^{+}/6^{-}, \qquad b : 21^{+}/9^{-}, \qquad c : 17^{+}/13^{-}.$$

For child target `c`, every positive example has $(A, B) = (1, 1)$. For root target `a`, the predictor cell $(B, C) = (0, 0)$ contains both labels:

Row	A (target)	B	C	Class for target `a`
9	1	0	0	positive
26	0	0	0	negative

Rows 9 and 26 therefore share the same BK literals `b_val_0` and `c_val_0` while carrying opposite labels for A. No deterministic rule over (B, C) separates them. Target `b` has the symmetric ambiguity on predictor cells involving (A, C).

3.4 Controlled extensions: trailing and leading distractors (H2, H3)

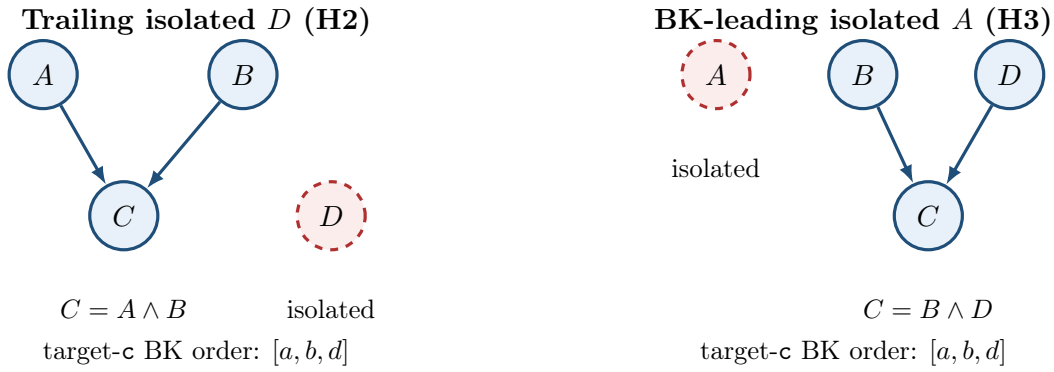


Figure 5: Controlled four-variable extensions. Dashed red nodes are independent isolated variables. Serialisation order induces the stated BK predictor order for target `c`.

Trailing isolated D (H2). This fixture preserves the H0 collider, the H0 root laws $A \sim$ Bernoulli(4/5) and $B \sim$ Bernoulli(7/10), and the mechanism $C = A \wedge B$. It adds an independent isolated root $D \sim$ Bernoulli(1/2) with no edges. Variable order places `d` last, so for target `c` the exact-value BK predictors appear as $[a, b, d]$: the distractor is trailing. The evaluator rule for `c` remains

```
c(A) :- a_val_1(A), b_val_1(A).
```

The variables A, B, and the isolated D are each generated stochastically. When any of them is the learning target, there is no underlying deterministic rule from the remaining observed variables that should be learned. The frozen table used below is again an IID sample with $n = 30$ and seed 42 on this four-variable process.

BK-leading isolated A (H3). This fixture uses independent roots

$$A \sim \text{Bernoulli}(1/2), \qquad B \sim \text{Bernoulli}(4/5), \qquad D \sim \text{Bernoulli}(7/10),$$

with A isolated (no edges) and deterministic child $C = B \wedge D$. Variable order places `a` first, so for target `c` the BK predictors appear as $[a, b, d]$: the distractor is leading. The evaluator rule for `c` is

```
c(A) :- b_val_1(A), d_val_1(A).
```

The isolated variable A and the roots B and D are each generated stochastically. When any of them is the learning target, there is no underlying deterministic rule from the remaining observed variables that should be learned. The frozen table used below is an IID sample with $n = 30$ and seed 42 on this process.

Both extensions have verified Markov factorisation and exact ordinary faithfulness certificates. As with the principal fixture, the generating graph, certificates, and mechanism references are withheld from the learner.

4 Experimental design and comparison logic

This section records the learner configurations used in the probes, how strongly each designed contrast supports attribution to a single change, and the ordered probe map. Detailed outcomes appear in the experimental sequence that follows.

4.1 Learner configurations

Identity	Semantics	Folding	Sel.	Space	Steps	`asm_intro`
`aamas2025`	brave	greedy	mgr	BK	—	`relto`
`ecai2024`	brave	nd	any	all	10	`relto`
`baseline_cautious`	cautious	nd	any	all	10	`relto`
`greedy_cautious`	cautious	greedy	mgr	BK	—	`relto`
`nd_brave_sechk`	brave	nd	any	all	10	`sechk`
`nd_cautious_sechk`	cautious	nd	any	all	10	`sechk`

Here "Steps" is `folding_steps`, which bounds the non-deterministic folding-token budget under nd folding. All six identities also enable the post-learning checked-ASP artefact (`check_ic`). The cautious nd identities additionally pin `post_folding_test_entailment(true)`, which is likewise the effective default under the brave configurations used here.

`aamas2025` and `ecai2024` are the repository presets corresponding to the AAMAS 2025 and ECAI 2024 learning-option bundles used in this work. `baseline_cautious`, `greedy_cautious`, `nd_brave_sechk`, and `nd_cautious_sechk` are experimental identities for controlled comparison. In particular, `greedy_cautious` matches `aamas2025` except for `learning_mode(cautious)`.

The H6a configurations match `baseline_cautious` except for `folding_steps`, with $M \in \{1, 2, 5, 10\}$. H7a matches `ecai2024` except `asm_intro(sechk)`. H7b matches `baseline_cautious` except `asm_intro(sechk)`.

4.2 Strength of comparisons

Each probe is read at one of three attribution strengths.

- **Exact paired option ablation.** Fixture, sample, encoding, and the remaining learner options are held fixed; one learning option changes. Probes H4 and H4b change `learning_mode` under otherwise fixed nd / `relto` settings; H5 changes `learning_mode` under the fixed Greedy / `mgr` / BK / `relto` bundle; H6a changes `folding_steps`; H7a and H7b change only `asm_intro`.

- **Controlled data or fixture intervention.** The learner configuration is held fixed (or compared only within one configuration); one designed property of the sample or fixture changes. H1 uses a nested seed-42 prefix $n = 25$ of the principal table under AAMAS; H2 adds a trailing isolated D under AAMAS; H3 uses the BK-leading isolated-A fixture under ECAI (with AAMAS as secondary); H6b varies nested sample size $n \in \{30, 60, 90\}$ at fixed `folding_steps(2)` under `baseline_cautious`.
- **Bundle-level contrast.** Several learner settings change together, so a behavioural difference is attributed to the configuration bundle rather than to one option. H0 compares `aamas2025` with `ecai2024` on the principal table. Where both exist, contrasts between `greedy_cautious` and `baseline_cautious` are likewise bundle-level (Greedy versus nd), as used secondarily in H5.

The strength level states how far a difference among outcomes, deltas, or traces can be attributed to the designed intervention of that probe.

4.3 Probe sequence

Probe	Question	Controlled intervention	What is inspected
H0	What do the Greedy and nd bundles learn for each target on the principal AND collider?	Principal fixture, frozen $n = 30$ seed 42; `aamas2025` and `ecai2024`; targets `a`,`b`,`c`	Exit status, deltas, and traces by target and configuration.
H1	How does omitting the rare $(0, 0, 0)$ atom affect AAMAS root learning?	Nested seed-42 prefix $n = 25$ versus $n = 30$; AAMAS; same stream	Root deltas and the presence or absence of opposite-label witness rows.
H2	What does Greedy do with an irrelevant trailing variable on the child?	Principal AND collider plus isolated D; AAMAS; target `c`	Whether `d` appears in the learned bodies relative to the D-free evaluator rule.
H3	What does nd learning do when an isolated variable is first in BK?	BD AND collider with BK-leading isolated A; order $[a, b, d]$; ECAI primary on target `c`	First fold, retained BK literals, and the final child theory.
H4	How does switching brave nd to cautious nd affect root targets on the principal table?	Learning mode only, under fixed nd / `relto`; targets `a`,`b` with control `c`	Root exit status and the control child delta.
H4b	How does the same brave-to-cautious switch affect the child repair on the BK-leading fixture?	Learning mode only on H3 target `c`; nd / `relto` fixed	Shared structure versus the contrary-rule close under `alpha_3`.
H5	How does brave-to-cautious behave inside the Greedy bundle?	`learning_mode` only under Greedy / `mgr` / BK / `relto`; 18 paired cells across the completed AAMAS collections	Paired outcomes, deltas, traces, and runtime; secondary contrast to nd cautious where available.
H6a	How does the folding-token ceiling affect cautious-nd search cost on roots?	`folding_steps` $M \in \{1, 2, 5, 10\}$; principal table $n = 30$; `baseline_cautious`	Trace length, runtime, token bands, and control child behaviour.

Probe	Question	Controlled intervention	What is inspected
H6b	How does grounded sample size affect the same cautious-nd search?	Nested $n \in \{30, 60, 90\}$ at fixed $M = 2$; principal fixture; `baseline_cautious`	Search topology and per-band growth with n.
H7a	How do `relto` and `sechk` differ under brave nd?	`asm_intro` only; principal and BK-leading fixtures; seven target pairs	Paired deltas and the assumption-introduction control-flow fork.
H7b	How do `relto` and `sechk` differ under cautious nd?	`asm_intro` only; same two fixtures; `baseline_cautious` versus `nd_cautious_sechk`	Child deltas, root search cost, and whether runs complete within the wall-time limit.

5 Experimental sequence: H0–H7b

5.1 H0: baseline on the principal AND collider

Question and purpose. On the principal fixture $A \to C \leftarrow B$ with $C = A \wedge B$, frozen sample $n = 30$ seed 42, and exact-value encoding, what ABA frameworks do the Greedy bundle (`aamas2025`) and the nd bundle (`ecai2024`) emit when each of `a`, `b`, and `c` is the learning target?

Controlled comparison. The fixture, table, encoding, and target-specific examples were held fixed. The two configurations differ as full bundles: `aamas2025` uses brave learning with Greedy / `mgr` / BK folding and `relto`; `ecai2024` uses brave learning with nd / `any` / `all` folding and `relto`. The contrast is therefore bundle-level.

Configuration	Target	Outcome	Emphasised result
AAMAS	`c`	`solved`	Assumption-free rule `c(A) :- a_val_1(A), b_val_1(A).`, string-identical to the evaluator mechanism.
AAMAS	`a`,`b`	`completed_no_solution`	Both-zero co-occurrence candidates cover opposite-label rows. Contrary repair dead-ends under `relto`.
ECAI	`c`	`solved`	First fold uses BK-leading `a_val_1`, repaired by an assumption attacked when `b_val_0`.
ECAI	`a`,`b`	`solved`	Brave per-row assumption construction on the ambiguous predictor cell. A (resp. B) remains underdetermined by the other observed variables.

Child target `c`

Emphasised observation. Under AAMAS, every positive example for `c` has $(A, B) = (1, 1)$. Greedy folding with BK space therefore builds the single maximal co-occurrence body $\{\texttt{a\_val\_1}, \texttt{b\_val\_1}\}$. After mgr deduplication the emitted delta is

```
c(A) :- a_val_1(A), b_val_1(A).
```

Brave entailment succeeds immediately: no negative example shares that body, so no assumption is introduced. The string coincides with the evaluator mechanism reference for C.

Under ECAI, nd folding takes one BK predicate at a time. From an early positive, the first fold replaces the example identifier by `a_val_1`, the first predictor in BK order, yielding the unary rule `c(A) :- a_val_1(A)`. That body also holds of every negative with $A = 1, B = 0$. Assumption introduction produces

```
c(A) :- alpha_1(A), a_val_1(A).
c_alpha_1(A) :- b_val_0(A).
assumption(alpha_1(A)).
contrary(alpha_1(A),c_alpha_1(A)) :- assumption(alpha_1(A)).
```

Trace-supported explanation. The ECAI theory is sample-adequate under brave checking: c is derived when $a = 1$, unless the assumption is attacked when $b = 0$. Across the target rule and the contrary it cites both parents of C. Its ABA shape is nevertheless different from the AAMAS conjunction: the first fold used the leading BK predictor, and the second parent enters only through contrary learning.

Root targets `a` and `b`

Emphasised observation. For target `a`, positives fall into two predictor cells, $(B, C) = (1, 1)$ and $(B, C) = (0, 0)$. The cell $(B, C) = (0, 0)$ also contains negatives: row 9 is positive with joint values $(1, 0, 0)$, while rows 26 and 30 are negative with $(0, 0, 0)$, yet all share the BK literals `b_val_0` and `c_val_0`. Target `b` is the symmetric case with the roles of A and B swapped.

Under AAMAS, greedy folding proposes the two Horn candidates

```
a(A) :- b_val_1(A), c_val_1(A).
a(A) :- b_val_0(A), c_val_0(A).
```

The both-one rule is accepted. The both-zero rule covers the negative rows 26 and 30, so brave entailment fails. The learner introduces an assumption on that body. Contrary rote learning then cites those negatives, and greedy folding of the contrary reconstructs the same both-zero body. Under `asm_intro(relto)` a further relative assumption fails for that body. The engine reports KO, and the cell ends as `completed_no_solution` with no emitted delta. Target `b` mirrors this path.

Under ECAI, nd folding again proceeds one predicate at a time. The learner first builds a safe block for the both-one positives,

```
a(A) :- alpha_1(A), b_val_1(A).
c_alpha_1(A) :- c_val_0(A).
```

That block covers positives with $(B, C) = (1, 1)$ and excludes negatives with $B = 1$ and $C = 0$. The both-zero positives remain uncovered, so a second ordinary target rule is still required.

Folding an early both-zero positive (row 8) produces the candidate `a(A) :- b_val_0(A)`. Brave entailment fails on that candidate. The same BK literals also hold of the negative examples in rows 26 and 30. The learner therefore introduces an assumption α_2 on the body, giving `a(A) :- alpha_2(A), b_val_0(A)`. Contrary rote learning cites rows 26 and 30 as grounds for `c_alpha_2`.

Folding that contrary first tries `b_val_0`. Under `relto` this returns KO against the already present α_2. The next fold tries `c_val_0`, yielding `c_alpha_2(A) :- c_val_0(A)`. Even that unary contrary remains too strong for the joint example set. The learner therefore introduces a further assumption α_3 on the contrary. Rote learning of `c_alpha_3` then cites the both-zero positives, including row 9. Folding that contrary re-uses α_2 together with `b_val_0`. The trace records this close as `found:`

`alpha_2/1 ... OK`. The final delta for target `a` is

```
a(A) :- alpha_1(A), b_val_1(A).
a(A) :- alpha_2(A), b_val_0(A).
c_alpha_1(A) :- c_val_0(A).
c_alpha_2(A) :- alpha_3(A), c_val_0(A).
c_alpha_3(A) :- alpha_2(A), b_val_0(A).
assumption(alpha_1(A)).
assumption(alpha_2(A)).
assumption(alpha_3(A)).
contrary(alpha_1(A),c_alpha_1(A)) :- assumption(alpha_1(A)).
contrary(alpha_2(A),c_alpha_2(A)) :- assumption(alpha_2(A)).
contrary(alpha_3(A),c_alpha_3(A)) :- assumption(alpha_3(A)).
```

Target `b` is the mirror obtained by swapping the roles of A and B.

Trace-supported explanation. The decisive geometry is the predictor cell $(B, C) = (0, 0)$. Row 9 is a positive example for target `a`. Its joint values are $(A, B, C) = (1, 0, 0)$. Row 26 is a negative example. Its joint values are $(0, 0, 0)$. Both rows carry the BK facts `b_val_0` and `c_val_0`. Any Horn rule that uses only those predictors therefore treats the two rows alike. It would derive `a` at both identifiers, or at neither.

The emitted α_2/α_3 fragment works because assumptions are grounded per row identifier. At a fixed row i, the relevant instances are $\alpha_2(i)$ and $\alpha_3(i)$, with contraries `c_alpha_2`(i) and `c_alpha_3`(i). The local rules are

```
a(i) :- alpha_2(i), b_val_0(i).
c_alpha_2(i) :- alpha_3(i), c_val_0(i).
c_alpha_3(i) :- alpha_2(i), b_val_0(i).
```

On every both-zero row, both `b_val_0(i)` and `c_val_0(i)` hold. Accepting $\alpha_2(i)$ therefore derives `c_alpha_3`(i) and attacks $\alpha_3(i)$. Accepting $\alpha_3(i)$ derives `c_alpha_2`(i) and attacks $\alpha_2(i)$. The two assumptions cannot both be accepted at the same row.

This leaves two locally stable choices at each both-zero identifier. If $\alpha_2(i)$ is accepted and $\alpha_3(i)$ is rejected, then `a(i)` is derived. If $\alpha_3(i)$ is accepted and $\alpha_2(i)$ is rejected, then `a(i)` remains underived.

Brave learning asks for one witnessing stable extension Δ of the whole grounded framework. That extension must cover every positive example and exclude every negative example at once. Such a witness exists. On row 9 it accepts $\alpha_2(9)$ and rejects $\alpha_3(9)$. The rule `a(9) :- alpha_2(9), b_val_0(9)` then derives the positive example $(1, 0, 0)$. On row 26 it accepts $\alpha_3(26)$ and rejects $\alpha_2(26)$. The same BK facts are present. Without $\alpha_2(26)$ the rule for `a` leaves `a(26)` underived, so the negative example $(0, 0, 0)$ is excluded. The two rows are therefore separated by their ground assumption atoms, even though their BK predictors are identical.

Row	Joint (A, B, C)	Choice in Δ	Effect
9	$(1, 0, 0)$ (positive)	$\alpha_2(9)$ in, $\alpha_3(9)$ out	`a(9)` derived
26	$(0, 0, 0)$ (negative)	$\alpha_2(26)$ out, $\alpha_3(26)$ in	`a(26)` blocked

Other stable extensions may reverse those local choices. Brave entailment requires only that at least one joint witness exists. The framework therefore passes brave checking on the finite sample.

For this mechanism, target `a` should have no learned solution: A is generated stochastically, and there is no underlying deterministic rule from the other observed variables. A solution was nev-

ertheless found. Brave entailment permits one stable extension to assign opposite statuses to the ground assumptions $\alpha_2(i)$ and $\alpha_3(i)$ at different row identifiers. That per-row split covers the positive example $(1, 0, 0)$ and excludes the negative example $(0, 0, 0)$ even though the two rows share the same BK predictors. The ECAI `solved` outcome records that brave covering of the finite examples under the selected semantics.

Evidential boundary. H0 is the establishing bundle contrast on one complete-support table. Procedural `solved` covers both the AAMAS mechanism-string match for `c` and the ECAI assumption-rich theories for `c` and the roots. Attribution of the configuration difference is to the full AAMAS versus ECAI option bundle.

Primary evidence record: `docs/experiments/qualitative/M13-C3-binary-collider-and/learning_analysis.md`

5.2 H1: nested prefix omitting the rare $(0, 0, 0)$ atom

Question and purpose. Under the fixed AAMAS bundle on the principal AND collider, what happens to the root targets when the seed-42 table is replaced by its nested prefix of length 25, which omits the only sample atoms $(0, 0, 0)$?

Controlled comparison. Everything except sample size is held fixed. H0 uses the first 30 rows of the seed-42 stream. H1 uses the exact first 25 rows of that same stream. The omitted rows are CSV rows 26 and 30, the only H0 occurrences of $(0, 0, 0)$. The population still assigns mass $P^*(0, 0, 0) = 3/50$ to that atom.

Target	H0 ($n = 30$)	H1 ($n = 25$)
`a`,`b`	`completed_no_solution`	`solved` with both-one and both-zero Horn rules
`c`	`solved` with the evaluator AND string	`solved` with the same AND string

Emphasised observation. The H1 root delta for target `a` is

```
a(A) :- b_val_1(A), c_val_1(A).
a(A) :- b_val_0(A), c_val_0(A).
```

Target `b` emits the mirror pair. The both-zero rules are false in the population. They are sample-adequate on H1 only because the prefix contains no negative example in the predictor cell $(B, C) = (0, 0)$ (resp. $(A, C) = (0, 0)$).

Trace-supported explanation. H0 already showed that, under AAMAS, the both-zero body is proposed for root `a` and is rejected when rows 26 and 30 are present as negatives in that cell. H1 removes exactly those rows. The same body then faces only positives in that cell, so brave entailment accepts it and the run ends as `solved`.

For these stochastic roots, the intended reading is that no deterministic rule from the other observed variables should be learned. The H0 outcome `completed_no_solution` matches that reading. The H1 outcome `solved` is a spurious finite-sample covering created by omitting the rare blocking atom. Child `c` is unchanged and serves only as a control.

Evidential boundary. H1 is a nested-prefix support ablation under fixed AAMAS settings.

Primary evidence record: `docs/experiments/qualitative/M13-C3-binary-collider-and/h1_support_ablation.md`

5.3 H2: trailing isolated variable under Greedy

Question and purpose. On the principal AND collider extended by an independent isolated root D, does AAMAS learning of the deterministic child `c` retain irrelevant `d` literals, or does it return the evaluator D-free rule `c(A) :- a_val_1(A), b_val_1(A).`?

Controlled comparison. The H0 collider, root laws, and mechanism $C = A \wedge B$ are preserved. An isolated root $D \sim \text{Bernoulli}(1/2)$ is added with no edges. Variable order places `d` last, so for target `c` the BK predictors are $[a, b, d]$. The run uses AAMAS on the frozen $n = 30$ seed-42 sample. Among the eighteen positive `c` rows, both $D = 1$ and $D = 0$ occur nine times.

Emphasised observation. AAMAS solves target `c` and emits

```
c(A) :- a_val_1(A), b_val_1(A), d_val_1(A).
c(A) :- a_val_1(A), b_val_1(A), d_val_0(A).
```

Irrelevant D is retained. The two rules are sample-adequate and jointly cover the observed values of D, but they are non-minimal relative to the evaluator mechanism

```
c(A) :- a_val_1(A), b_val_1(A).
```

On the three-variable H0 table without D, the same AAMAS bundle emitted exactly that D-free string.

Trace-supported explanation. Under greedy folding with `folding_space(bk)`, each positive example is folded into a maximal co-occurrence body from the BK literals that hold of that row. A positive with $D = 1$ therefore keeps `d_val_1`. A positive with $D = 0$ keeps `d_val_0`. The trace shows this explicitly: after `a_val_1` and `b_val_1`, the next fold attaches the matching `d_val_*` literal. Mgr selection then keeps the resulting bodies. There is no later step that merges the complementary pair by deleting the `d` literal. The failure mode under test is therefore distractor retention on a solved child, not failure to solve `c`.

Evidential boundary. H2 is a controlled fixture intervention under fixed AAMAS settings. The primary signal is the balanced $n = 30$ distractor split on target `c`.

Primary evidence record: `docs/experiments/qualitative/M13-C3-binary-collider-and/h2_irrelevant_covariate.md`

5.4 H3: BK-leading isolated variable under nd

Question and purpose. On a BD AND collider with an isolated root placed first in BK order, does ECAI learning of the deterministic child `c` select that isolated variable in the first fold and retain it in the final theory, or does it return the evaluator rule `c(A) :- b_val_1(A), d_val_1(A).`?

Controlled comparison. The fixture is $B \to C \leftarrow D$ with $C = B \wedge D$ and an independent isolated root $A \sim \text{Bernoulli}(1/2)$. Variable order places `a` first, so for target `c` the BK predictors are $[a, b, d]$. The primary run is ECAI brave nd on the frozen $n = 30$ seed-42 sample. An AAMAS run on the same table supplies a bundle-level contrast. Among the fifteen positive `c` rows, every row has $(B, D) = (1, 1)$, while both values of A occur.

Emphasised observation. The ECAI trace opens with the first fold

```
folding result: c(A) <- [a_val_1(A)]
```

Assumption introduction follows immediately. The full emitted delta for target `c` is:

```
c(A) :- alpha_1(A), a_val_1(A).
c(A) :- alpha_2(A), a_val_0(A).
c_alpha_1(A) :- b_val_0(A).
c_alpha_1(A) :- alpha_3(A), b_val_1(A).
c_alpha_2(A) :- d_val_0(A).
c_alpha_2(A) :- b_val_0(A).
c_alpha_3(A) :- alpha_1(A), a_val_1(A).
assumption(alpha_1(A)).
assumption(alpha_2(A)).
assumption(alpha_3(A)).
contrary(alpha_1(A),c_alpha_1(A)) :- assumption(alpha_1(A)).
contrary(alpha_2(A),c_alpha_2(A)) :- assumption(alpha_2(A)).
contrary(alpha_3(A),c_alpha_3(A)) :- assumption(alpha_3(A)).
```

The two ordinary target rules are conditioned only on value literals of the isolated variable A (together with assumptions α_1 and α_2). Mechanism parents enter through contrary rules: `c_alpha_1` and `c_alpha_2` cite b and d, and under the a=1 branch one contrary rule of α_1 contains a further assumption α_3. The theory is therefore organised around A, with B and D used to repair over-general A-conditioned target rules. The evaluator reference for this target remains `c(A) :- b_val_1(A), d_val_1(A).`.

On the same sample, AAMAS also solves `c` while retaining `a`:

```
c(A) :- a_val_1(A), b_val_1(A), d_val_1(A).
c(A) :- a_val_0(A), b_val_1(A), d_val_1(A).
```

Trace-supported explanation. Under nd folding with `any` selection, the first admissible BK predicate becomes the first generalising fold. Here that predicate is an `a`-value literal, because `a` leads the serialised BK. The unary fold over-generalises, so the learner introduces α_1 (then α_2) and learns contraries for those A-conditioned target rules. Later folds place b and d in `c_alpha_1` and `c_alpha_2`, but the ordinary target rules keep the initial a-value literals. The AAMAS contrast reaches distractor retention by a different route: greedy maximal co-occurrence keeps every matching literal, including leading `a`, and splits the AND across the two values of A.

In both cases the child is solved, and in both cases the isolated leading variable is retained. The failure mode is distractor inclusion driven by BK order.

Contrary-structure divergence under the two A-conditioned target rules. The ECAI delta is asymmetric under the two ordinary target rules conditioned on a=1 and a=0. That asymmetry is part of the H3 theory and deserves a separate reading.

Under a=1 (α_1), the contrary of α_1 is *assumption-bearing*: one contrary rule contains a further

assumption α_3:

```
c_alpha_1(A) :- b_val_0(A).
c_alpha_1(A) :- alpha_3(A), b_val_1(A).
c_alpha_3(A) :- alpha_1(A), a_val_1(A).
```

Under a=0 (α_2), the contrary of α_2 is *ordinary*: every contrary rule uses only BK literals, citing the mechanism parents directly:

```
c_alpha_2(A) :- d_val_0(A).
c_alpha_2(A) :- b_val_0(A).
```

So the a=1 side is repaired by a further assumption α_3, while the a=0 side is repaired by ordinary b/d literals alone.

The divergence is a search-order effect under nd folding with `any` selection, b-before-d BK order after leading `a`, cover-and-repair, and `asm_intro(relto)`. The underlying mechanism of C is the same on both sides of the split. The same assumption-bearing α_1 contrary and ordinary α_2 contrary appear under both H3 brave and H4b cautious. Concretely:

1. Contrary repair for `c_alpha_1` occurs first, because the first positive example has a=1.
2. While repairing that contrary, the first contrary seed has $(b, d) = (0, 1)$, so search commits to `b_val_0`. The remaining contrary residuals have $(b, d) = (1, 0)$.
3. Folding those residuals with `b_val_1` over-generalises. Cover-and-repair introduces α_3 on the `b_val_1` body.
4. When search later repairs `c_alpha_2` under a=0, a fold that would reuse `b_val_1` meets the already declared α_3 under `relto` and is rejected. Search then falls through to `d_val_0` and `b_val_0`, yielding the ordinary BK-only contrary.

Thus the assumption-bearing α_1 contrary and the ordinary α_2 contrary are two stages of one sequential repair path. The a=0 side inherits the assumption minted while repairing the a=1 side.

Evidential boundary. H3 is a controlled fixture intervention. The primary signal is ECAI first-fold retention of BK-leading isolated A on target `c`. The assumption-bearing versus ordinary contrary split under α_1 and α_2 is a secondary procedural feature of that same theory.

Primary evidence record: `docs/experiments/qualitative/M13-C3-binary-collider-and/h3_bk_leading_distractor.md`

5.5 H4: brave versus cautious under nd

5.5.1 H4: roots on the H0 table

Question and purpose. On the principal AND-collider table of H0, does changing `learning_mode(brave)` to `learning_mode(cautious)` alter the outcomes for root targets `a` and `b` that, under `ecai2024`, relied on a residual per-row assumption construction separating rows 9 and 26, and what happens to control target `c`?

Controlled comparison. The H0 fixture, frozen $n = 30$ seed-42 table, target-specific examples, and BK were held fixed. The two learner configurations compared are `ecai2024` and `baseline_cautious`. Both use nd folding with `folding_selection(any)`, `folding_space(all)`, `asm_intro(relto)`, and `folding_steps(10)`. They differ in learning mode: `ecai2024` sets `learning_mode(brave)`; `baseline_cautious` sets `learning_mode(cautious)`. The locked `ecai2024` outputs are the brave comparator.

Target	ecai2024	baseline_cautious
a	solved	completed_no_solution
b	solved	completed_no_solution
c	solved	solved, identical delta

Emphasised observation. For target a, the ecai2024 and baseline_cautious traces coincide through the early $\alpha_1, \alpha_2, \alpha_3$ construction from H0. They diverge at one assumption-introduction step. After

```
folding result: c_alpha_3(A) <- [b_val_0(A)]
```

the learner proposes to re-use the already declared assumption α_2 in that body:

```
c_alpha_3(A) :- alpha_2(A), b_val_0(A).
```

Under ecai2024 the proposal is accepted; under baseline_cautious it is rejected:

```
% ecai2024:
 found: alpha_2/1 ... OK, assumption introduction result:
   c_alpha_3(A) <- [alpha_2(A),b_val_0(A)]

% baseline_cautious:
 found: alpha_2/1 ... KO, cannot introduce an assumption for [b_val_0(A)]
```

Under ecai2024, that OK completes the both-zero assumption–contrary fragment from H0 and the learner writes a solution. Under baseline_cautious, the KO is followed by further failed folds, exhaustion of folding_steps(10), and completed_no_solution with no emitted delta. Target b exhibits the same divergence with the roles of A and B swapped.

Under baseline_cautious, control target c remains solved. Its delta is identical to the locked ecai2024 delta for c:

```
c(A) :- alpha_1(A), a_val_1(A).
c_alpha_1(A) :- b_val_0(A).
assumption(alpha_1(A)).
contrary(alpha_1(A),c_alpha_1(A)) :- assumption(alpha_1(A)).
```

Thus learning_mode(cautious) still admits an assumption-bearing theory for the child on this table, while rejecting the residual α_2 reuse for the roots.

Trace-supported explanation. Under both configurations the search builds toward the same both-zero assumption–contrary fragment:

```
a(A) :- alpha_2(A), b_val_0(A).
c_alpha_2(A) :- alpha_3(A), c_val_0(A).
c_alpha_3(A) :- alpha_2(A), b_val_0(A).
```

Write $\mathcal{F}_{\text{bz}}$ for a framework containing this fragment. The decisive conflict is observational. Row 9 contributes $a(9) \in E^+$ with joint values $(A, B, C) = (1, 0, 0)$. Row 26 contributes $a(26) \in E^-$ with joint values $(0, 0, 0)$. Relative to BK the two rows are indistinguishable: both carry b_val_0 and c_val_0. No Horn body over those predictors alone can accept one row and reject the other.

The fragment separates the rows by ground assumption membership rather than by BK. At any

both-zero row identifier i, the assumptions $\alpha_2(i)$ and $\alpha_3(i)$ attack each other through their contrary rules, so a stable extension can take only one of the two local choices:

Choice at row i	Acceptance of $a(i)$	Derived contrary
$\alpha_2(i) \in \Delta$, $\alpha_3(i) \notin \Delta$	$\mathcal{F}_{\text{bz}} \models_\Delta a(i)$	`c_alpha_3`(i)
$\alpha_2(i) \notin \Delta$, $\alpha_3(i) \in \Delta$	$\mathcal{F}_{\text{bz}} \not\models_\Delta a(i)$	`c_alpha_2`(i)

Brave entailment. Under `learning_mode(brave)`, a learned framework $\mathcal{F}'$ is accepted when there exists one witnessing stable extension covering the examples:

$$\exists \Delta \in \text{Stab}(\mathcal{F}') \left[\forall e \in E^+,\ \mathcal{F}' \models_\Delta e \ \wedge\ \forall e \in E^-,\ \mathcal{F}' \not\models_\Delta e\right].$$

For $\mathcal{F}_{\text{bz}}$ such a witness exists. On row 9 it chooses $\alpha_2(9)$ in and $\alpha_3(9)$ out, so

```
a(9) :- alpha_2(9), b_val_0(9)
```

yields $\mathcal{F}_{\text{bz}} \models_\Delta a(9)$. On row 26 it chooses $\alpha_2(26)$ out and $\alpha_3(26)$ in, so $\mathcal{F}_{\text{bz}} \not\models_\Delta a(26)$. The closing rule

```
c_alpha_3(A) :- alpha_2(A), b_val_0(A).
```

is part of making those local choices coherent: whenever $\alpha_2(i)$ is accepted, `c_alpha_3`(i) is derived and attacks $\alpha_3(i)$.

Thus $\mathcal{F}_{\text{bz}}$ passes brave entailment because coverage may depend on different ground assumption choices at different row identifiers inside a single witness. The assumption-introduction check that proposes re-using α_2 on `c_alpha_3(A) <- [b_val_0(A)]` therefore returns OK, and `ecai2024` finishes with `solved`.

Failure under cautious entailment. Under `learning_mode(cautious)`, the same framework must satisfy the stronger condition

$$\forall e \in E^+,\ \mathcal{F}' \models_{\text{caut}} e, \qquad \forall e \in E^-,\ \mathcal{F}' \not\models_{\text{caut}} e,$$

where $\mathcal{F}' \models_{\text{caut}} s$ means that s is accepted with respect to every stable extension of $\mathcal{F}'$.

Apply that test to $\mathcal{F}_{\text{bz}}$ at row 9. The choice that covers the positive is $\alpha_2(9) \in \Delta$. The opposite choice, $\alpha_3(9) \in \Delta$ and $\alpha_2(9) \notin \Delta$, also belongs to $\text{Stab}(\mathcal{F}_{\text{bz}})$. In any extension Δ' that takes the opposite choice, $\mathcal{F}_{\text{bz}} \not\models_{\Delta'} a(9)$. Therefore $a(9)$ fails $\mathcal{F}_{\text{bz}} \models_{\text{caut}} a(9)$. The brave cover of the positive exists as an existential cover. Cautious entailment requires universal cover, and $\mathcal{F}_{\text{bz}}$ leaves $a(9)$ undecided across stable extensions on this underdetermined BK cell.

That is why, under `baseline_cautious`, the same proposal returns

```
found: alpha_2/1 ... KO, cannot introduce an assumption for [b_val_0(A)]
```

The learner rejects the clause that would close the brave construction. Search then exhausts `folding_steps(10)` and returns `completed_no_solution`.

In short, `ecai2024` succeeds because brave entailment needs only some stable extension that assigns opposite α_2/α_3 statuses to rows 9 and 26. Under `baseline_cautious`, $a(9)$ fails to hold in every stable extension of $\mathcal{F}_{\text{bz}}$, so the residual α_2 reuse fails the cautious entailment check.

The control on target `c` is consistent with these semantics. Its contrary folds to the ordinary body `b_val_0`. That body treats rows that share the same BK literals uniformly, so the cautious entailment checks succeed, and the emitted delta matches the `ecai2024` delta for `c`.

Evidential boundary. H4 compares learner configurations `ecai2024` and `baseline_cautious` on the locked H0 table, with matched nd folding options, matched `asm_intro(relto)`, and `folding_steps(10)`. Under `baseline_cautious` the roots end as `completed_no_solution`.

Primary evidence record: `docs/experiments/qualitative/M13-C3-binary-collider-and/h4_cautious_vs_brave.md`

5.5.2 H4b: child c on the H3 table

Question and purpose. On the H3 BD AND table with BK-leading isolated A, under matched nd and `relto` settings, how does changing `learning_mode(brave)` to `learning_mode(cautious)` alter the contrary of α_3 when learning target `c`?

Controlled comparison. The H3 fixture `m13_bucket3_binary_bd_and_lead_a`, frozen $n = 30$ seed-42 sample, target `c`, and BK order $[a, b, d]$ were held fixed. The two learner configurations compared are `ecai2024` and `baseline_cautious`. Both use nd folding with `folding_selection(any)`, `folding_space(all)`, `asm_intro(relto)`, and `folding_steps(10)`. They differ in learning mode: `ecai2024` sets `learning_mode(brave)`; `baseline_cautious` sets `learning_mode(cautious)`. The locked H3 `ecai2024` cell for target `c` is the brave comparator.

Object	`ecai2024`	`baseline_cautious`
Outcome for `c`	`solved`	`solved`
Ordinary target rules	$\alpha_1, a{=}1$ / $\alpha_2, a{=}0$	same
`c_alpha_1`	$b{=}0$; also $\alpha_3, b{=}1$	same
`c_alpha_2`	$d{=}0$; $b{=}0$	same
`c_alpha_3`	$\alpha_1, a{=}1$	$d{=}1$

Emphasised observation. Both configurations solve target `c`. Through the introduction of α_3, their theories coincide with the H3 theory already recorded: ordinary target rules conditioned on leading a, an assumption-bearing contrary under $a{=}1$, and an ordinary b/d contrary under $a{=}0$. Exactly one closing rule differs. Under `ecai2024`:

```
c_alpha_3(A) :- alpha_1(A), a_val_1(A).
```

Under `baseline_cautious`:

```
c_alpha_3(A) :- d_val_1(A).
```

The traces diverge at the assumption-introduction check on that close. After

```
folding result: c_alpha_3(A) <- [a_val_1(A)]
```

`ecai2024` re-uses α_1 and stops:

```
found: alpha_1/1 ... OK, assumption introduction result:
  c_alpha_3(A) <- [alpha_1(A),a_val_1(A)]
```

`baseline_cautious` rejects that reuse, rejects a subsequent attempt to re-use α_3 on `b_val_1`, and then accepts an ordinary `d_val_1` fold:

```
found: alpha_1/1 ... KO, cannot introduce an assumption for [a_val_1(A)]
```

```
folding result: c_alpha_3(A) <- [b_val_1(A)]
found: alpha_3/1 ... KO, cannot introduce an assumption for [b_val_1(A)]
folding result: c_alpha_3(A) <- [d_val_1(A)]
```

The full `baseline_cautious` delta is therefore the shared H3 theory with that single close replaced:

```
c(A) :- alpha_1(A), a_val_1(A).
c(A) :- alpha_2(A), a_val_0(A).
c_alpha_1(A) :- b_val_0(A).
c_alpha_1(A) :- alpha_3(A), b_val_1(A).
c_alpha_2(A) :- d_val_0(A).
c_alpha_2(A) :- b_val_0(A).
c_alpha_3(A) :- d_val_1(A).
assumption(alpha_1(A)).
assumption(alpha_2(A)).
assumption(alpha_3(A)).
contrary(alpha_1(A),c_alpha_1(A)) :- assumption(alpha_1(A)).
contrary(alpha_2(A),c_alpha_2(A)) :- assumption(alpha_2(A)).
contrary(alpha_3(A),c_alpha_3(A)) :- assumption(alpha_3(A)).
```

Trace-supported explanation. The shared prefix is the H3 theory under leading a. As recorded in H3, that prefix is produced by nd folding with `any` selection, b-before-d BK order, cover-and-repair, and `asm_intro(relto)`. It is reproduced under both learning modes through α_3 introduction. The assumption-bearing versus ordinary contrary split under α_1 and α_2 is therefore already present before the learning-mode contrast takes effect.

The learning-mode contrast is confined to the attack on α_3. Under `ecai2024`, the learner closes that attack by residual reuse of α_1:

```
c_alpha_3(A) :- alpha_1(A), a_val_1(A).
```

This clause couples the contrary of α_3 to the same assumption that appears in the ordinary target rule under a=1. Brave entailment asks only for some witnessing stable extension of the candidate framework that covers E^+ and excludes E^-. That residual reuse passes the brave assumption-introduction check, so `ecai2024` accepts it and finishes without folding a d-literal on `c_alpha_3`.

Under `baseline_cautious`, the entailment condition is universal: every positive must be a cautious consequence, and every negative must be excluded as a cautious consequence. The residual reuse of α_1 on `c_alpha_3` fails that test, exactly as the residual reuse of α_2 failed under H4: acceptance of the proposed extension depends on assumption choices that fail to hold in every stable extension. The introduction check therefore returns KO. On this H3 child cell an ordinary BK alternative remains available. Folding the contrary examples of α_3 with `d_val_1` satisfies the cautious entailment condition on $\langle E^+, E^- \rangle$, so the learner accepts

```
c_alpha_3(A) :- d_val_1(A).
```

and finishes with `solved`.

Relative to H4, the same learning-mode change therefore has the same local effect of rejecting a residual assumption reuse, together with a different global outcome. On the H0 roots, search exhausts `folding_steps(10)` without an accepted covering theory. On this H3 child cell, an ordinary parent literal closes the contrary of α_3, while the ordinary target rules conditioned on leading a and the asymmetric contrary structure from H3 remain in place.

Evidential boundary. H4b compares `ecai2024` and `baseline_cautious` on the locked H3 target-`c` cell, with matched nd folding options and `asm_intro(relto)`. The emitted theories retain BK-leading isolated A in the ordinary target rules. The learning-mode effect isolated here is the body of `c_alpha_3`.

Primary evidence record: `docs/experiments/qualitative/M13-C3-binary-collider-and/h4b_cautious_split_under_a.md`

5.6 H5: Greedy under cautious versus brave learning

Question and purpose. Under otherwise fixed Greedy options, does changing `learning_mode(brave)` to `learning_mode(cautious)` alter the accepted or rejected theories on the completed Greedy collections from H0–H3?

Controlled comparison. The locked `aamas2025` collections for H0 ($n = 30$), H1 ($n = 25$), H2 ($n = 30$), the extreme H2 edge ($n = 2$), and H3 ($n = 30$) were re-run under the experimental configuration `greedy_cautious`. That configuration matches `aamas2025` on Greedy folding, `folding_selection(mgr)`, `folding_space(bk)`, `asm_intro(relto)`, and `check_ic`, and differs only by setting `learning_mode(cautious)`. This gives eighteen target-wise pairs on shared tables, BK, and examples. Configuration `greedy_cautious` is an experimental preset for this contrast.

Emphasised observation. On all eighteen pairs, `greedy_cautious` matches `aamas2025` on every outcome, and every solved delta is identical. The only systematic difference is runtime: cautious runs take approximately 1.6–2.3 times as long as the corresponding brave runs (mean ratio about 1.84), while remaining below one second.

Trace-supported explanation. Accepted and rejected Greedy theories are identical across the learning-mode change on these cells. The runtime increase comes from the entailment checks performed along the shared search path. Brave checks ask whether some answer set satisfies the joint example constraints. Cautious checks ask Clingo for cautious consequences (`-enum-mode=cautious`), that is atoms true in every answer set, and then test E^+ and E^- against that intersection. The same sequence of checks is therefore more expensive under cautious learning.

Evidential boundary. H5 is restricted to these eighteen Greedy pairs under matched non-mode options.

Primary evidence record: `docs/experiments/qualitative/M13-C3-binary-collider-and/h5_greedy_cautious.md`

5.7 H6: explain cautious-nd search cost

5.7.1 H6a: folding-token ceiling

Question and purpose. On the H0 table under `baseline_cautious`, how do root no-solution trace length and runtime scale with the folding-token ceiling $M =$ `folding_steps`?

Controlled comparison. The H0 fixture, frozen $n = 30$ seed-42 sample, target-specific examples, BK, `learning_mode(cautious)`, nd folding, `folding_selection(any)`, `folding_space(all)`, and `asm_intro(relto)` were held fixed. The single varied option was `folding_steps(M)` for

$M \in \{1, 2, 5, 10\}$. Configurations `baseline_cautious_steps1`, `baseline_cautious_steps2`, and `baseline_cautious_steps5` match `baseline_cautious` except for M. The $M = 10$ collection reuses the locked H4 run under `baseline_cautious`. Roots `a` and `b` are the intended no-solution targets (stochastic; no desired deterministic rule from the other observed variables). Child `c` is the control.

M	Target	Outcome	Runtime (s)	Trace lines	N_{asm}
1	a	`completed_no_solution`	6.638	549	9
2	a	`completed_no_solution`	13.117	1047	18
5	a	`completed_no_solution`	32.775	2541	45
10	a	`completed_no_solution`	69.952	5031	90

Here N_{asm} is the number of stdout lines matching `generating NEW assumption:` ... in one target-wise Prolog trace (the literal engine message; "NEW" is part of that string). It counts assumption-*introduction events*, not the number of assumptions that remain in an accepted delta.

Target b mirrors these figures. Control `c` solves at every M inside `tokens(1)`: 134-line trace, about 1.3 s, and a identical delta across all four ceilings.

Emphasised observation. For roots `a` and `b` the outcome is `completed_no_solution` at every tested M: no covering theory is accepted. Each run is a stack of M failed bands. The $M = 1$ trace has 549 lines and $N_{\text{asm}} = 9$. Each extra allowance adds 498 lines and nine further such events. At $M = 10$ the trace prints nine restarts

```
* Increasing folding tokens to: 2
...
* Increasing folding tokens to: 10
```

and ends with `* No solution found!`. The counts fit

$$\text{lines}(M) = 549 + 498(M - 1) = 51 + 498\,M, \qquad N_{\text{asm}}(M) = 9M.$$

Runtime grows in the same way. Raising M multiplies failed bands. Every tested ceiling ends as `completed_no_solution`. The long H4 root traces are the $M = 10$ endpoint of this ladder.

Trace-supported explanation. A failed band is one complete cautious search at fixed token allowance T that exhausts without writing a delta. On these roots the H4 residual assumption reuse already returns the engine message `KO, cannot introduce an assumption for` ... inside the first allowance, so that first band fails.

Under nd folding, `folding_steps(M)` is the ceiling on how many such attempts may run. Search starts at `tokens(1)`. When the attempt at allowance T fails completely and $T + 1 \leq M$, the controller prints `* Increasing folding tokens to: \(T{+}1\)` and restarts the same learning problem on the same rules and the same E^+, E^-. The target still admits no cautiously acceptable cover, so each restart fails in the same way. M allowances therefore contribute M copies of the same failed band, which is why trace length and N_{asm} are linear in M.

Control `c` solves under `tokens(1)` and is therefore invariant in M.

Evidential boundary. H6a is a single-option ablation of `folding_steps` under `baseline_cautious` on the locked H0 $n = 30$ table, for these intended no-solution roots over $M \in \{1, 2, 5, 10\}$.

5.7.2 H6b: nested sample size

Question and purpose. On the H0 table under `baseline_cautious` with folding-token ceiling fixed at $M = 2$, how do root no-solution trace length and runtime scale when the frozen seed-42 sample is enlarged by nested prefixes $n \in \{30, 60, 90\}$?

Controlled comparison. The H0 fixture, seed 42, target-specific encoding, and learner configuration `baseline_cautious_steps2` were held fixed: `learning_mode(cautious)`, nd folding, `folding_selection(any)`, `folding_space(all)`, `asm_intro(relto)`, and `folding_steps(2)`. The single varied factor was sample size: nested prefixes $n = 30 \subset n = 60 \subset n = 90$ of the same seed-42 draw. Roots `a` and `b` are the intended no-solution targets; child `c` is the control. The $n = 30$ cell reuses the H6a $M = 2$ collection.

n	Target	Outcome	Runtime (s)	Trace lines	N_{asm}	N_{KO}
30	a	`completed_no_solution`	13.117	1047	18	32
60	a	`completed_no_solution`	22.169	1641	18	32
90	a	`completed_no_solution`	32.291	2241	18	32

Here N_{asm} is as in H6a: the count of stdout lines `generating NEW assumption:` .... N_{KO} is the count of stdout lines `KO, cannot introduce an assumption for` .... Both are event counts from one target-wise Prolog trace. They are included as a check that the *sequence of assumption-introduction and failed residual reuse events* is the same at every n, while trace length and runtime may still grow.

Target b mirrors these figures. Every root run prints a single restart to `tokens(2)`. Control `c` solves at every n with a identical delta; its trace grows only from 134 to 200 to 266 lines (about 1.3–3.0 s).

All four H0 population atoms are already present at $n = 30$. Their multiplicities, ordered as $(0,0,0), (0,1,0), (1,0,0), (1,1,1)$, are

$$n = 30 : (2, 4, 7, 17),$$
$$n = 60 : (4, 10, 12, 34),$$
$$n = 90 : (5, 17, 18, 50).$$

Emphasised observation. Root outcomes are invariant in n: always `completed_no_solution`. The assumption-event counts are likewise invariant: $N_{\text{asm}} = 18$ and $N_{\text{KO}} = 32$ at every tested n, with one increase to `tokens(2)`. That matches H6a at $M = 2$, where each failed band contributes nine `generating NEW assumption` events, so two bands contribute $N_{\text{asm}} = 9M = 18$. What grows is how much text each band prints: target-a trace length rises by 594 then 600 lines for each additional 30 rows, and runtime rises somewhat faster than lines. Control `c` keeps the same accepted theory while its short trace lengthens mildly with n.

Trace-supported explanation. H6a varies how many failed bands may run. H6b holds that band count fixed at two allowances and varies how much grounded work sits inside each band. The fixed N_{asm} and N_{KO} show that the learner still takes the same pattern of assumption-introduction events and failed residual reuses; only the volume of grounding work between those events changes. Additional IID rows add example identifiers, rote facts, entailment and subsumption checks, and contrary batches. Support presence is already complete at $n = 30$, and only multiplicities change. The near-linear growth in trace length is therefore per-row grounding and bookkeeping cost. Runtime grows a little faster because ASP consequence checks scale with that larger grounding.

Evidential boundary. H6b is a nested-n ablation at fixed $M = 2$ under `baseline_cautious` on the locked H0 seed-42 prefixes $n \in \{30, 60, 90\}$. It measures cost when multiplicities change while assumption-event counts N_{asm} and N_{KO} stay fixed. The $n \times M$ grid was left for later work.

Primary evidence record: `docs/experiments/qualitative/M13-C3-binary-collider-and/h6_folding_and_n_ablation.md`

5.8 H7: change assumption introduction from `relto` to `sechk`

H7 holds the nd folding bundle fixed and changes only `asm_intro`. H7a uses brave learning. H7b uses cautious learning. The two probes therefore isolate the same control-flow fork under the two entailment modes already contrasted in H4/H4b.

5.8.1 H7a: brave nd

Question and purpose. Under brave nd on the locked H0 and H3 $n = 30$ tables, if only `asm_intro` changes from `relto` to `sechk`, which BK predictors appear in the accepted deltas?

Controlled comparison. Fixture, frozen seed-42 sample, target, examples, BK, `learning_mode(brave)`, nd folding, `folding_selection(any)`, `folding_space(all)`, `folding_steps(10)`, and `check_ic` were held fixed. Locked `ecai2024` uses `asm_intro(relto)`. Experimental `nd_brave_sechk` matches it except for `asm_intro(sechk)`. Seven pairs: H0 `a`,`b`,`c` and H3 `a`,`b`,`c`,`d`.

Emphasised observation. Both configurations solve all seven cells. Every paired `delta.aba` is distinct. The emphasised pattern is on the `sechk` side: in all seven `sechk` deltas, the only observed BK predictor that appears is the *first* predictor in that target's BK order. Later predictors are absent. Matched `relto` deltas cite that first predictor and also later ones:

Cell	BK order	Predictors in `sechk` delta	Extra predictors in `relto` delta
H0 a	b, c	b	c
H0 b	a, c	a	c
H0 c	a, b	a	b
H3 a	b, c, d	b	c, d
H3 b	a, c, d	a	c, d
H3 c	a, b, d	a	b, d
H3 d	a, b, c	a	b, c

For target `a` the first BK predictor is b. Illustrative H0 child `c` (BK order a, b):

```
% ecai2024 / relto
c(A) :- alpha_1(A), a_val_1(A).
c_alpha_1(A) :- b_val_0(A).

% nd_brave_sechk / sechk
c(A) :- alpha_1(A), a_val_1(A).
c_alpha_1(A) :- alpha_2(A), a_val_1(A).
c_alpha_2(A) :- alpha_1(A), a_val_1(A).
```

Both share `c(A) :- alpha_1(A), a_val_1(A)`. Under `relto`, the contrary of α_1 is `b_val_0`. Under `sechk`, every BK literal in the delta is an a-value literal.

Trace-supported explanation. After a fold fails post-folding entailment, the policies fork in `gen.pl`:

```
relto:  gen2: ... looking for assumption relative to
sechk:  gen2: generating NEW assumption: alpha_1(A)
```

Under `relto`, a residual reuse that returns `KO, cannot introduce an assumption for` ... can hand control back to folding, so a later BK literal (for example `b_val_0` on H0 `c`) may be tried next. That is how later predictors enter the `relto` deltas.

Under `sechk`, a provisional assumption is minted immediately and search continues through `gen5`/`gen6`. On these brave cells, the accepted theory is completed using only value literals of the first BK predictor, before those later folds are reached. Hence every H7a `sechk` delta cites exactly one observed BK predictor: the first in that target's BK order.

Evidential boundary. H7a is a single-option `asm_intro` ablation under brave nd on these seven pairs. The cautious counterpart is H7b.

Primary evidence record: `docs/experiments/qualitative/M13-C3-binary-collider-and/h7a_relto_vs_sechk.md`

5.8.2 H7b: cautious nd

Question and purpose. Under cautious nd on the locked H0 and H3 $n = 30$ tables, if only `asm_intro` changes from `relto` to `sechk`, which BK predictors appear in the accepted deltas for child `c`?

Controlled comparison. Fixture, frozen seed-42 sample, target `c`, examples, BK, `learning_mode(cautious)`, nd folding, `folding_selection(any)`, `folding_space(all)`, `folding_steps(10)`, `post_folding_test_entailm` and `check_ic` were held fixed. Locked `baseline_cautious` uses `asm_intro(relto)`. Experimental `nd_cautious_sechk` matches it except for `asm_intro(sechk)`. The lead pairs are H0 `c` and H3 `c`.

Emphasised observation. Both configurations solve both child cells. Every paired `delta.aba` is distinct. On both tables, the cautious `sechk` delta cites the first BK predictor and also later predictors. That is the emphasised contrast with H7a, where every brave `sechk` delta cited only the first BK predictor.

For H0 `c` (BK order a, b):

```
% baseline_cautious / relto
c(A) :- alpha_1(A), a_val_1(A).
c_alpha_1(A) :- b_val_0(A).

% nd_cautious_sechk / sechk
c(A) :- alpha_1(A), a_val_1(A).
c_alpha_1(A) :- alpha_2(A), a_val_1(A).
c_alpha_2(A) :- b_val_1(A).
```

Both share `c(A) :- alpha_1(A), a_val_1(A)`. Under `relto`, the contrary of α_1 is `b_val_0`. Under `sechk`, the contrary of α_1 contains a further assumption α_2 conditioned on `a_val_1`, and the contrary of α_2 is `b_val_1`. Parent b therefore appears in both deltas. The closing b-literals differ (`b_val_0` versus `b_val_1`), and `sechk` inserts one extra assumption between the shared target rule and that b-literal.

For H3 `c` (BK order a, b, d), both deltas keep ordinary target rules conditioned on leading a and place parents b and d in contrary rules. The `sechk` delta has five assumptions against three under `relto`, and its trace is longer (899 lines against 257).

Trace-supported explanation. The policies fork as in H7a after a fold fails post-folding entailment:

```
relto:  gen2: ... looking for assumption relative to
sechk:  gen2: generating NEW assumption: alpha_1(A)
```

Under cautious learning, residual reuse often returns `KO, cannot introduce an assumption for ...` in `gen3`. On the `relto` path, that KO returns control to folding, so a later BK literal such as `b_val_0` on H0 `c` can be tried next. On the `sechk` path, a provisional assumption is installed first and search continues through `gen5`/`gen6`. When an early candidate that uses only a-literals fails cautious checking, further assumption introduction and backtracking still reach later b or d predicates. That is why these cautious `sechk` deltas for `c` include mechanism parents, while the brave H7a `sechk` deltas for `c` used only the first BK predictor.

The same mint-first path accounts for the longer H3-`c` `sechk` trace. More provisional repairs are attempted before later folds succeed.

Evidential boundary. H7b is a single-option `asm_intro` ablation under cautious nd on the locked H0 and H3 child-`c` cells, with `baseline_cautious` as the `relto` comparator.

Primary evidence record: `docs/experiments/qualitative/M13-C3-binary-collider-and/h7b_cautious_relto_vs_sechk.md`

6 Findings

6.1 Finding 1: a completed Greedy solution enumerates the distinct positive value patterns

Call the *value pattern* of a row the conjunction listing every non-target column at the value that row takes. Under the exact-value encoding, a completed Greedy solution is exactly the set of distinct value patterns of the positive rows, one rule for each, with no assumptions and no contraries. Every learned rule therefore cites every predictor available in the background knowledge, including predictors the evaluator knows to be isolated from the target.

The transformations force this result. Rote learning creates one rule per positive row whose body is that row's identifier. Greedy folding under `folding_space(bk)` walks that body and accumulates every background head matching the row, and the encoding supplies exactly one value predicate per non-target column per row, so the folded body is the row's complete value pattern. Greedy folding is deterministic, so the body is a function of the row. Selection under `folding_selection(mgr)` and subsumption then discard whole rules that repeat a pattern, and the transformation set available to the learner contains no step that deletes a literal from a body, so the surviving rules are the distinct positive patterns and nothing smaller.

Checking this against every recorded Greedy cell confirms it. Of the 22 cells, which include four from an earlier fixture, the learner ran in 20 and solved 8. In all 8, the set of rule bodies in `delta.aba` equals the set of distinct positive value patterns computed from that cell's own `data.csv`, every body cites every predictor column, and no delta contains an assumption or a contrary.

The same mechanism decides whether a solution exists at all. A rule whose body is a full value

pattern is satisfied by exactly the rows carrying that pattern, so it covers a negative example precisely when some negative row repeats a positive row's pattern. Greedy therefore needs no assumption when the positive and negative pattern sets are disjoint, and it has no sound rule available when they intersect. The recorded outcomes divide on exactly that line. All 8 solved cells have disjoint pattern sets. All 12 `completed_no_solution` cells share at least one pattern between the two labels. The two remaining cells are the $n = 2$ tables, which contain no negative example and were skipped before learning.

The consequence for causal reading is that an irrelevant column is retained whenever it is observed, and it multiplies the theory. H2 places an isolated D last in background-knowledge order and H3 places an isolated A first, and each solves its child while retaining that variable, so order is immaterial here. The H0 child needs the single rule `c(A) :- a_val_1(A), b_val_1(A).`, which coincides with its evaluator mechanism. The H2 fixture attaches an isolated D to that same child mechanism, and its child theory is the two rules recorded under H2, one for each observed value of D. The positive rows there carry two distinct value patterns where H0's carry one, and the delta reports both.

Beyond this finding. The counting is what should generalise. With k isolated binary predictors, each observed at both values among the positive rows, the distinct positive patterns of a deterministic child multiply by 2^k, so the Greedy delta should hold 2^k rules while the mechanism it describes stays fixed. H2 is the $k = 1$ case, where one rule became two. The size of the learned theory therefore tracks the width of the table, and the number of value combinations available to be memorised grows exponentially in the columns that the mechanism ignores.

We take this tentatively to make Greedy unsuitable for causal recovery under the exact-value encoding. Adding columns refines value patterns, which can only reduce the chance that a positive and a negative row share one. Once the number of combinations is large relative to the sample size, almost every positive row carries a pattern of its own, no negative row repeats it, and the disjointness condition above is met for reasons that have nothing to do with the mechanism. Greedy would then return a completed solution for a target whose value the other observed variables do not determine, and that solution would be the positive rows restated as rules. Whether a target receives a theory at all would depend on how many irrelevant variables happen to be measured.

6.2 Finding 2: nd latches onto the first background-knowledge predictor

Every completed nd solution latches onto the predictor whose block comes first in target-specific background knowledge. The ordinary rules for the target cite that one observed variable and no other, with one rule for each value it takes among the positive rows, and each rule pairs the value literal with a single assumption. The remaining variables enter through the assumptions and their contraries, which is how nd represents what it has learned, so a delta is read as a whole and not through its base rules alone. What background-knowledge order fixes is the variable the base rules latch onto.

The latch forms on the first fold. Background knowledge is serialised one predictor block per column, nd folding begins with a single folding token, and `folding_selection(any)` accepts the first admissible fold, so the leading predictor is folded first and one token admits one value literal:

```
 folding result: c(A) <- [a_val_1(A)]
gen2: extended ABA does not entail <E+,E-> - looking for assumption relative to
gen2: generating NEW assumption: alpha_1(A)
```

The body $c \leftarrow a_1$ covers negative examples, and the learner responds by minting an assumption to block them. The predicate chosen for the fold is never reconsidered. Because the repair machinery

can rescue whichever fold was attempted first, the enumeration has no occasion to advance to the second predictor, and the learned structure accumulates in the contrary of the new assumption instead. The same sequence repeats for a_0 and yields the second target rule with α_2.

Initial folding follows background-knowledge position and takes no account of mechanistic relevance, so an isolated variable is latched onto whenever it is placed ahead of the mechanism parents. In H3 the child mechanism is $B \wedge D$, the isolated A leads the target-c order, and the solution latches onto a. In H0 the child mechanism is $A \wedge B$ with order $[a, b]$, and the solution latches onto a, which is a parent. The same position delivers an isolated variable in one fixture and a parent in the other. H0 is a clear case, because A and B enter its mechanism symmetrically as conjuncts and the latch breaks that symmetry in the direction background knowledge supplies. The single target rule

```
c(A) :- alpha_1(A), a_val_1(A).
```

is identical in all nine cells that solve this target, across seven configurations and three sample sizes.

What varies across those nine cells is the repair beneath the initial folding. Under `relto` the contrary is `c_alpha_1(A) :- b_val_0(A).`, which brings the second parent back into the theory one level down. Under cautious `sechk` the parent enters one level deeper again, through `c_alpha_2(A) :- b_val_1(A).` Under brave `sechk` the variable b appears nowhere in the delta. The composition of the repair chain therefore responds to the configuration.

Beyond this finding. The first admissible fold is determined by the order in which predictor blocks are serialised, and the repair machinery rescues that fold instead of rejecting it, so whichever predictor is placed first should be the one a completed solution latches onto. The same data presented under different orders should yield theories latched onto different variables. A causal reading has to account for this, because a theory latched onto an isolated variable and a theory latched onto a parent are indistinguishable by outcome, and the difference between them is settled by column order before any mechanism variable is examined.

6.3 Finding 3: brave nd accepts theories for targets that have no mechanism, and cautious nd refuses them

Brave nd reports `solved` for targets whose values no function of the observed predictors can reproduce. The theory it accepts separates rows carrying identical predictor values by giving ground assumptions opposite statuses at those rows. Brave acceptance requires one stable extension that covers the sample, and such an extension is available whenever the conflicting rows can be assigned different assumption statuses. The `solved` outcome therefore records that a covering was found, and it carries no information about whether the target depends on the observed predictors.

H0 target a is the clearest instance. The learner receives b and c as predictors, and nine rows of the sample carry $b{=}0, c{=}0$, of which seven are labelled positive and two negative. No function of b and c reproduces both labels. This is the pattern overlap of Finding 1, and Greedy returns `completed_no_solution` on this table for that reason. Brave nd returns `solved`.

The decisive part of the delta is a pair of assumptions whose contraries refer to each other:

```
a(A) :- alpha_2(A), b_val_0(A).
c_alpha_2(A) :- alpha_3(A), c_val_0(A).
c_alpha_3(A) :- alpha_2(A), b_val_0(A).
```

In the ASP form the learner checks, an assumption holds when its contrary fails:

```
alpha_2(A) :- not c_alpha_2(A), b_val_0(A).
alpha_3(A) :- not c_alpha_3(A), c_val_0(A).
```

At a row where b=0 and c=0, both contrary rules are live, so α_2 holds when α_3 fails and α_3 holds when α_2 fails. The two assumptions attack each other at that row, which admits two stable states, one placing a in the extension and one leaving it out. Every literal is indexed by the row identifier A, so the choice is made independently at each conflicting row, and one extension can derive a at the seven positive rows and leave it underived at the two negative rows despite their identical predictor values.

Cautious learning refuses this construction. It requires each example to hold in every stable extension, and the opposite assignment at a conflicting row is stable too, so the positive examples there fail the check. The two runs diverge at the step that closes the mutual attack. The brave run accepts the reuse that produces `c_alpha_3(A) :- alpha_2(A), b_val_0(A).`, the cautious run rejects it with `KO, cannot introduce an assumption`, and both H0 roots end as `completed_no_solution`. Cautious learning remains able to accept a theory where one is warranted. On the same collections it accepts the child target, whose value is a function of its predictors, and emits a delta byte-identical to the brave one.

The pattern holds across every run recorded. Brave nd reported `solved` for all 14 target-wise cells whose target carries rows of this kind, spanning three fixtures, both `relto` and `sechk` assumption introduction, and conflicting-row counts from 4 to 48. Cautious nd accepted none of the 10 it attempted, returning `completed_no_solution` on the H0 roots and reaching `timeout` on the wider fixture. Across those runs cautious learning withheld a theory for every target with no mechanism over the observed variables and left the target that has one unchanged.

This is a failure mode of brave learning under nd. A brave `solved` outcome cannot be read as evidence that a mechanism over the observed variables exists, because the assumption machinery supplies a covering either way.

6.4 Finding 4: the folding-token ceiling repeats one search attempt

Under the exact-value target-wise encoding, every folding call completes after a single folding step. The option `folding_steps(M)` therefore controls how many times the learner restarts the same learning problem after a failed attempt, and it leaves the set of folded rules the learner can reach unchanged. Targets consequently divide in two. A target that receives an accepted theory receives it under `tokens(1)`, and M leaves its delta, trace, and runtime unchanged. A target that receives no accepted theory produces M identical search attempts, so M multiplies the cost of establishing the same `completed_no_solution` outcome.

The mechanism is legible in two consecutive trace lines. The folding predicate prints the tokens remaining before each step, and prints `DONE` once no atom is left to be folded:

```
2: folding [A=1] with b_val_1(_7934): b_val_1(A) <- [A=1]
1: DONE
```

Completion at 1 after a step begun at 2 records one token spent out of two available. The shape of the encoded input is what forces this. A rote rule offered to folding carries one row-identifier equality in its body, `a(A) <- [A=1]`, and the background rule used to fold it carries the same single equality, `b_val_1(A) :- A=1`. The step consumes that equality and contributes no further atom to be folded, so the call completes having spent one token whatever the allowance was.

This is a property of the encoding, so it holds across targets, fixtures, and learner options. Auditing every nd trace recorded in this investigation (49 target-wise cells, brave and cautious learning, `relto` and `sechk`, $n \in \{30, 50, 60, 90\}$) shows every folding call spending exactly one token, every list of atoms offered to folding holding exactly one atom, and no occurrence of the engine's token-

exhaustion diagnostic.

Because folding behaves identically at every allowance, a restart reproduces its predecessor exactly. H6a shows this on the H0 roots, where trace length follows $51 + 498M$ at all of $M \in \{1, 2, 5, 10\}$ and the control child `c` is unchanged. The $M = 10$ trace for target `a` is a 51-line preamble followed by ten attempts of 497 lines each, and those ten attempts are identical once generated assumption indices, Prolog variable numbers, and the printed token counter are normalised. The same identity holds in all fourteen recorded traces that raised the allowance, where one attempt ranges from 497 to 9747 lines, so the repetition is indifferent to what a single attempt costs.

The other half of the dichotomy is equally uniform. All 30 solved nd cells finished inside `tokens(1)`, so `folding_steps(1)` would have produced every accepted theory reported in this document.

Beyond this finding. The finding is conditional on the encoding, which raises the question of when a budget above one is operative at all. A second token is required exactly when a folding step leaves atoms still to be folded. In `fold_nd_wtc/7` that arises when the rule used for folding carries body atoms beyond the atom being folded, because the unmatched remainder is appended to the atoms still to be folded. The exact-value encoding supplies only rules of the form `b_val_1(A) :- A=1`, which carry no such remainder.

The conjecture is therefore that `folding_steps(2)` and `folding_steps(3)` become operative, in the sense that they change which theories the learner can reach, as soon as the background knowledge contains rules whose bodies conjoin more than one atom. A probe would hold the causal fixture, the frozen sample, and the learner options fixed, extend the background knowledge with defined predicates of that shape, and compare $M = 1, 2, 3$. Two observables would decide it. The diagnostic `0: FAIL - No more folding tokens left`, absent from every run recorded here, should appear at $M = 1$. Folded bodies accepted at $M = 2$ or $M = 3$ should be unreachable at $M = 1$, which would make the ceiling a genuine bound on folding depth for the first time.

Which defined predicates to add is a design decision that has not been taken. It governs whether such a probe measures folding depth alone or also enlarges the hypothesis space available to the learner, and the two readings would support different conclusions.

6.5 Finding 5: the search is fixed by the sample's distinct value patterns

What determines the learner's search is the set of distinct joint value patterns present in the sample. The multiplicity of those patterns fixes how much grounded bookkeeping that search performs. Enlarging a sample that already exhibits every pattern therefore leaves the search path and the learned theory exactly as they were, and raises only cost.

H6b holds the folding ceiling at $M = 2$ and enlarges the H0 sample through nested seed-42 prefixes $n \in \{30, 60, 90\}$. All four population atoms are present already at $n = 30$, so the prefixes change multiplicities alone, from $(2, 4, 7, 17)$ to $(5, 17, 18, 50)$ over the assignments $(0, 0, 0), (0, 1, 0), (1, 0, 0), (1, 1, 1)$.

Across those three samples the learner takes an identical sequence of decisions. Collecting every `gen1`–`gen6` step, folding result, assumption introduction, entailment check, and token increase in the order printed, and normalising Prolog variable numbers and row identifiers, root target `a` takes the same 238 decisions at $n = 30$, $n = 60$, and $n = 90$. Target `b` likewise takes 238 and child `c` takes 14. Summed over every line category whose count is invariant, the trace holds exactly 513 lines for each root and 63 for the child at all three sample sizes.

Growth is confined to five categories, all of them rote-rule creation and subsumption bookkeeping. Counting target-`a` lines after replacing assumption indices by `N` and row identifiers by `_`:

```
                                                    n=30   n=60   n=90
```

```
* subsumed: deleted!                                  192   432   672
evaluating subsumption of a(A) <- [A=_]               166   342   518
ert: c_alpha_N(A) <- [A=_]                             96   188   286
evaluating subsumption of c_alpha_N(A) <- [A=_]        56   120   184
ert: a(A) <- [A=_]                                     24    46    68
```

The last row is the clearest case. Rote learning creates one rule per positive row, so its count is the multiplicity of the positive assignments: 24, 46, and 68 at the three sample sizes. Child `c` shows the same correspondence, where `ert: c(A) <- [A=_]` appears 17, 34, and 50 times, exactly the multiplicity of $(1, 1, 1)$. Each duplicate row therefore contributes a rote rule, the subsumption evaluations that dispose of it, and the grounded entailment work attached to it, while contributing nothing to the decisions the learner takes. Trace length for target `a` rises by 594 and then 600 lines per additional 30 rows, and runtime rises from 13.1 to 22.2 to 32.3 seconds, somewhat faster than line count because the ASP consequence checks scale with the size of the grounding.

The same encoding is acutely sensitive to which patterns occur. Under the Greedy bundle, H1 removes the only two rows carrying $(0, 0, 0)$ from this stream, and the root outcome moves from `completed_no_solution` to `solved` with a population-false rule, because the predictor cell $(B, C) = (0, 0)$ then holds positives alone. Deleting one pattern changes the learned theory. Adding sixty rows that repeat existing patterns changes none of it.

6.6 Finding 6: `sechk` confines a brave solution to one variable and makes a cautious search longer

Changing `asm_intro` from `relto` to `sechk` has two effects. Under brave learning, every `sechk` delta cites exactly one observed variable, the first in background-knowledge order, in its target rules and its contrary definitions alike. Under cautious learning, `sechk` returns the same outcome as `relto` on every cell and, where it produces a theory, names the same variables, with a search between 1.5 and 3.5 times longer.

The brave result holds on all seven H7a pairs. Each `sechk` delta is built entirely from value literals of one variable, while each matched `relto` delta cites every predictor available to the target, two of two on the H0 cells and three of three on the H3 cells. On the H0 child, whose mechanism is $A \wedge B$ and whose background-knowledge order is $[a, b]$:

```
c(A) :- alpha_1(A), a_val_1(A).
c_alpha_1(A) :- alpha_2(A), a_val_1(A).
c_alpha_2(A) :- alpha_1(A), a_val_1(A).
```

Parent B appears nowhere. The matched `relto` delta shares the target rule and gives α_1 the contrary `b_val_0`, which names B. A theory confined to one variable cannot express a dependence on two parents, so this brave `sechk` run could not have reported the mechanism whatever the data contained. This is the Finding 2 latch with no exit, because the assumption chain closes on the variable it started from.

Under cautious learning the outcome and the variables named are unchanged, and the search is longer on every cell where both settings terminate:

Cell	Outcome (both)	relto		sechk	
		lines	s	lines	s
H0 a	completed_no_solution	5030	69.9	13730	167.6
H0 b	completed_no_solution	5117	66.5	13817	172.9
H0 c	solved	134	1.3	200	2.2
H3 c	solved	257	2.9	899	10.1

On the two solved cells the variable sets are identical, a, b and a, b, d, and what sechk adds is assumptions, two against one and five against three.

The search is longer because sechk has no cheap way to reject a repair. When a fold fails the entailment test, relto first tries an assumption already present in the framework, and a candidate that fails the test is abandoned for the price of one entailment check, with the framework unchanged. A sechk run has one response to a failed fold, which is to create an assumption, and it pays for the creation in full. The assumption's rules are built, the stable extensions of the extended framework are enumerated and then iterated over, its contrary is rote-learned, and folding resumes over the enlarged rule set. Cautious entailment has to hold in every stable extension, so most repairs are rejected, and under sechk almost every rejection carries that price. The rules rote-learned along the way supply further folds, which fail in turn and call for further assumptions.

The counts follow that account. Both cells below are H7b pairs listed in the table above. On H0 target b, a root of the collider, where the two settings work through the whole token ladder and report completed_no_solution, relto abandoned 160 repairs at the reuse stage and created 90 assumptions, while sechk created 620. Those extra assumptions bring extra folds with them, 740 attempted under sechk against 300 under relto. H3 target c, the child of b and d, moves the same way on a smaller scale, with 3 assumptions and 14 folds under relto against 48 assumptions and 58 folds under sechk.

The cost separation is specific to cautious learning. Under brave learning a covering theory is accepted after a few repairs, and all seven cells finish in under two seconds on either setting. Under cautious learning the longer sechk search returns the same verdict over the same variables, which makes relto the setting to prefer.

7 Implications

The implications below are design recommendations for subsequent work. They are drawn from Findings 1–5 and from the H0–H7b probe evidence.

7.1 Prefer nd folding over Greedy for mechanism-oriented learning

Finding 1 shows that, under the exact-value encoding, Greedy retains every observed predictor that is present on a positive row. An isolated variable is therefore written into the child theory whenever it is measured, and the number of emitted rules tracks the number of distinct positive value patterns rather than the mechanism.

That same mechanism produces spurious solutions for targets that have no deterministic rule from the other variables. Finding 1 shows that Greedy returns solved exactly when no negative row repeats a positive row's full BK value pattern, and that each accepted rule is one such positive pattern. A pattern lists every non-target column. Adding further binary columns therefore makes each pattern more specific: two rows that agreed on the previous columns can disagree on a new one and cease to share a pattern. As the number of columns grows relative to the sample size, the

chance that any negative row matches any positive pattern falls. In the limit almost every positive row carries a unique pattern, positive and negative pattern sets are disjoint for that reason alone, and Greedy accepts one rule per positive row. The outcome is `solved`, and the delta is the positive examples restated as rules, even though the predictors do not determine the target. Spurious completion is therefore expected whenever enough irrelevant columns are measured relative to n.

Nd folding has a structural advantage for mechanism recovery. A completed nd delta need not cite every observed column. Ordinary target rules and contrary repair can be confined to a subset of the predictors, so an irrelevant variable can remain absent from the theory. Greedy has no such option under this encoding: co-presence on a positive row forces inclusion.

The design implication is therefore to prefer nd folding for mechanism recovery. Greedy is inappropriate for that goal because it must retain every co-present predictor and, as the table widens with irrelevant columns, can complete by restating positive rows as rules. Nd is preferable because its solutions have the potential to omit irrelevant variables. Background-knowledge order still affects which predictors nd cites first.

7.2 Prefer cautious entailment under nd

Brave nd can return `solved` for stochastic root targets that the evaluator reference treats as having no deterministic rule from the other observed variables. Under H0 and H4, root `a` is solved by a construction that separates rows sharing identical BK literals by assigning opposite ground assumption statuses in a single witnessing stable extension. Finding 3 is the cross-cutting statement of that pattern. Cautious entailment rejects that residual reuse on the same tables, and the roots end as `completed_no_solution`. The design implication is to run nd learning under `learning_mode(cautious)`.

7.3 Compress the ABALearn table to distinct value patterns

Finding 5 shows that the learner's decision sequence is fixed by the set of distinct joint value patterns in the sample. Multiplicities of those patterns add rote facts, subsumption work, and grounded entailment checks, and they leave the emitted theory and the normalised search path unchanged. For no-solution roots under cautious nd, that bookkeeping cost grows approximately linearly with sample size at fixed folding ceiling.

A natural pipeline therefore separates two uses of the sample. Population-level quantities that Causal ABA-style reasoning would consume, such as estimates of dependence, independence, and related association structure, are computed from the full IID table, where frequencies matter. Once those quantities have been extracted, the table passed to target-wise ABALearn should retain one row per distinct joint value pattern.

7.4 Cap the folding-token ceiling

Finding 4 shows that, under the exact-value encoding used throughout this investigation, every folding call spends exactly one token. For targets that receive an accepted theory, that theory is already reached under `tokens(1)`, so raising `folding_steps(M)` leaves the delta unchanged. For targets that receive no accepted theory, each increment of M replays the same failed attempt, and cost grows linearly with M.

The immediate design implication for the present encoding is therefore to set the ceiling at the smallest value that preserves reachability of the theories already obtained. On the recorded evidence that value is $M = 1$.

Looking ahead, a bound is still needed when background knowledge admits multi-atom fold bodies and a second token can change which theories are reachable. A working proposal for discussion is to cap M at the number of background-knowledge predictors available to the target, that is $V - 1$ when there are V observed variables and one is the learning target. That proposal assumes a sufficient folding depth for unary and shallow multi-literal repairs. Finding 4 establishes the exact-value case, where only $M = 1$ affects reachability. The $V - 1$ cap is offered as a concrete default to test once richer background predicates make larger allowances operative.

8 Conclusion

The working recommendation from this investigation is to treat cautious nd learning with `relto` assumption introduction as the default configuration for accepting target-wise theories under the exact-value encoding studied here. Nd folding avoids the all-predictor retention that Finding 1 attributes to Greedy. Cautious entailment blocks the brave root solutions recorded under H0 and H4 and collected as Finding 3. Together they give the configuration already used as `baseline_cautious`, with the folding ceiling and table representation tightened as above.

Two further controls should sit in front of that learner. First, dependence and related association structure extracted by Causal ABA-style methods can be used to order columns and rows and to derive the background knowledge whose first predictor nd will place in the ordinary target rules. Finding 2 makes that ordering consequential. Second, after those population-level quantities have been read from the full sample, ABALearn should run on the support-compressed table and under a folding-token ceiling no larger than needed. On the present encoding that ceiling is $M = 1$. Under richer background knowledge the working proposal is a cap of $V - 1$.

The next step is to inspect what the existing Causal ABA implementation actually computes, and then to design how its intermediate or final outputs can supply ordering, background knowledge, and related constraints to target-wise ABALearn.